Architectural Diagram and Data Flow Report

Langfuse traces -> canonical episodes -> PostgreSQL JSONB -> Apache AGE -> memory-assisted ticket execution

Primary finding

The repository contains an implemented closed-loop design in opencode_orchestrator_postgres_age.py. It retrieves prior episodes before planning, traces plan/build execution in Langfuse, distills the trace into a validated Episode, stores raw and compact forms in PostgreSQL, and projects selected relationships into Apache AGE.

System scope

- Requirements: local Jira and Confluence fixtures
- Execution: OpenCode plan and build agents
- Observability: Langfuse SDK plus OpenTelemetry plugin
- Distillation: schema-constrained episode extraction LLM
- Canonical storage: PostgreSQL memory.agent_traces and memory.episodes
- Relationship storage: Apache AGE property graph
- Feedback: hybrid structural and PostgreSQL full-text retrieval

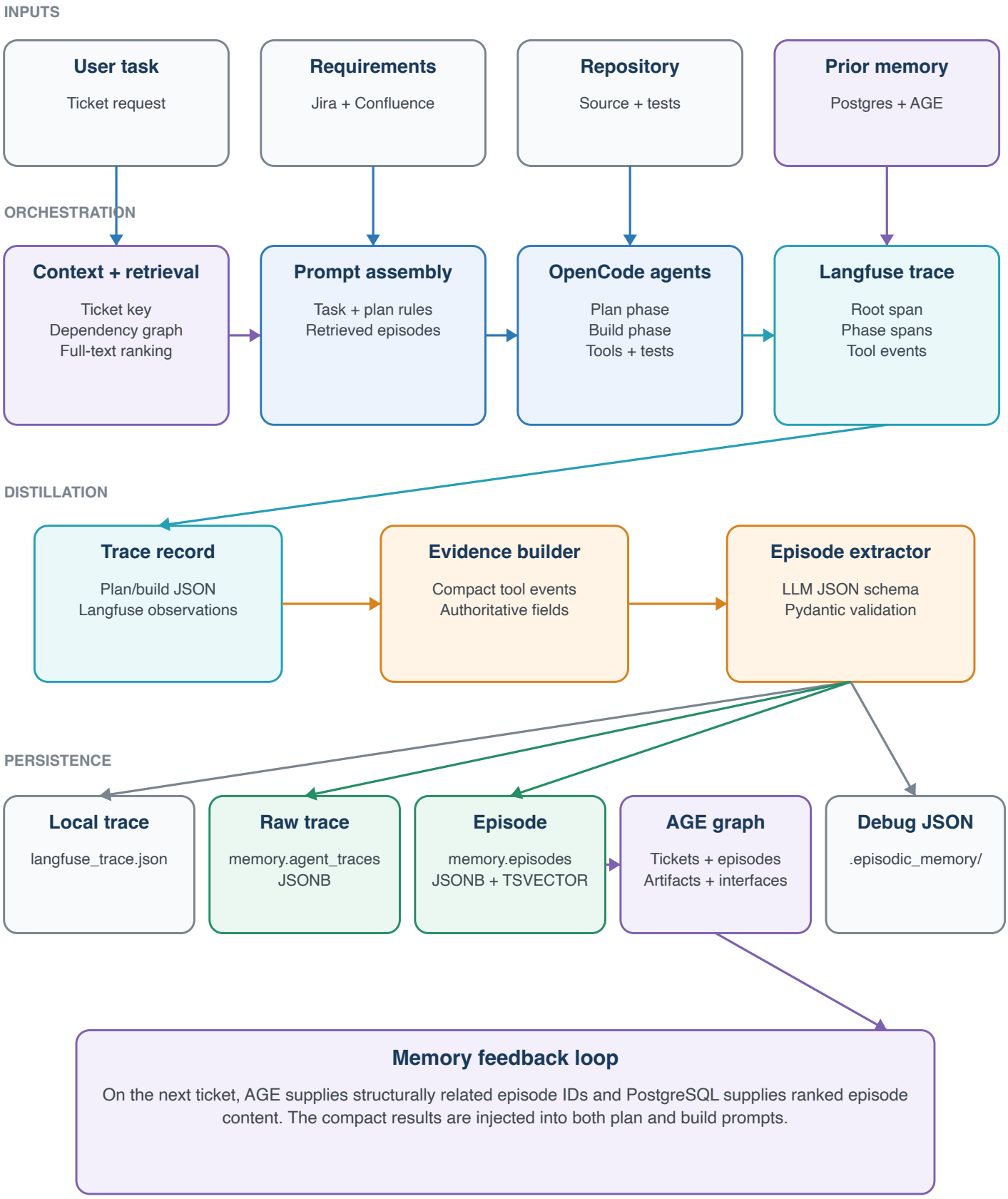
Current integration risk

The active loaders expect jira_tickets.json and confluence_stories.json, while this repository stores individual files under jira/ and confluence/. The AGE dependency graph can therefore remain empty and memory retrieval may fall back to text search.

Primary entry point: opencode_orchestrator_postgres_age.py :: main()

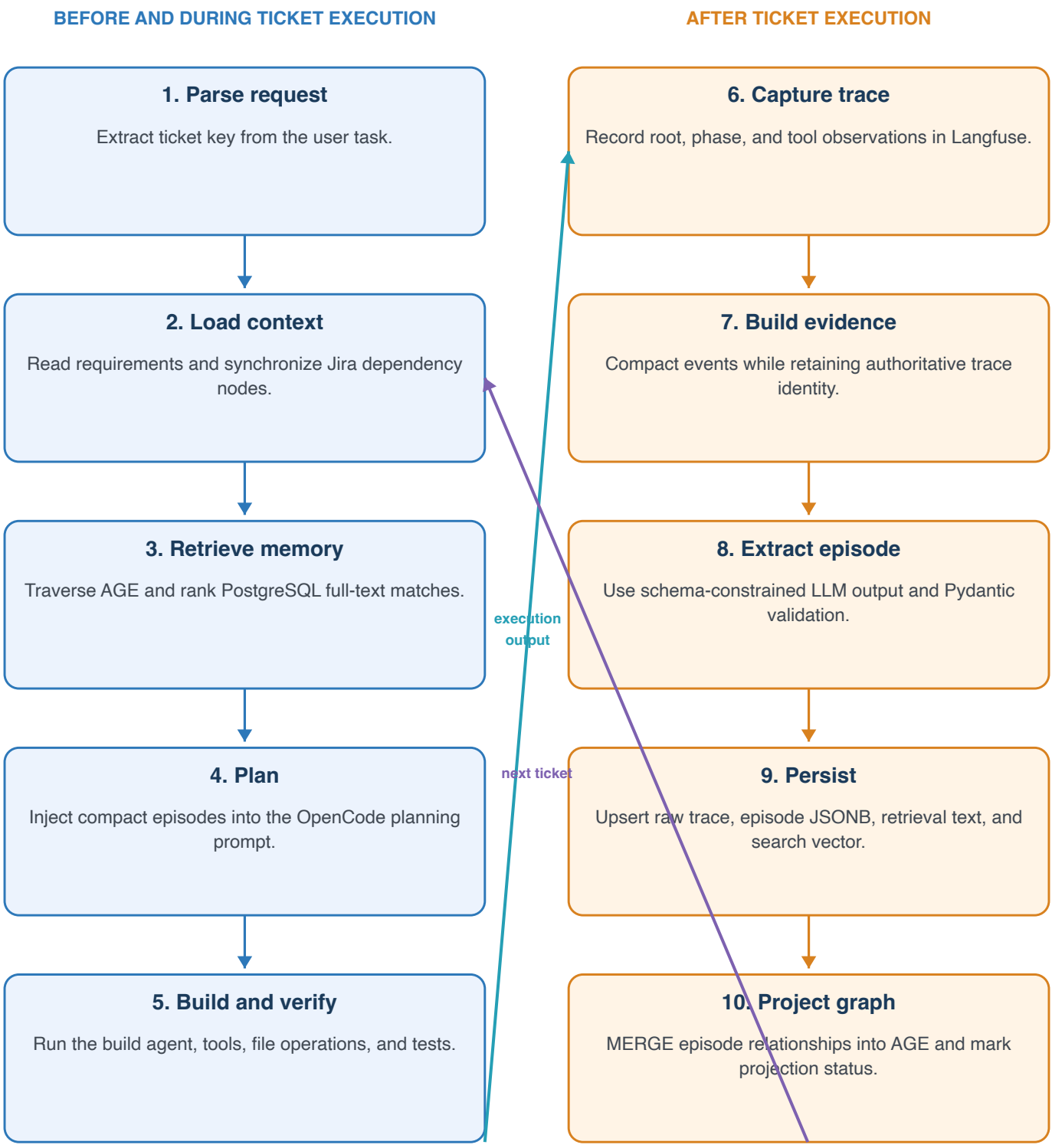
System Architecture

The PostgreSQL/AGE orchestrator is the only path that closes both the write and retrieval loops.



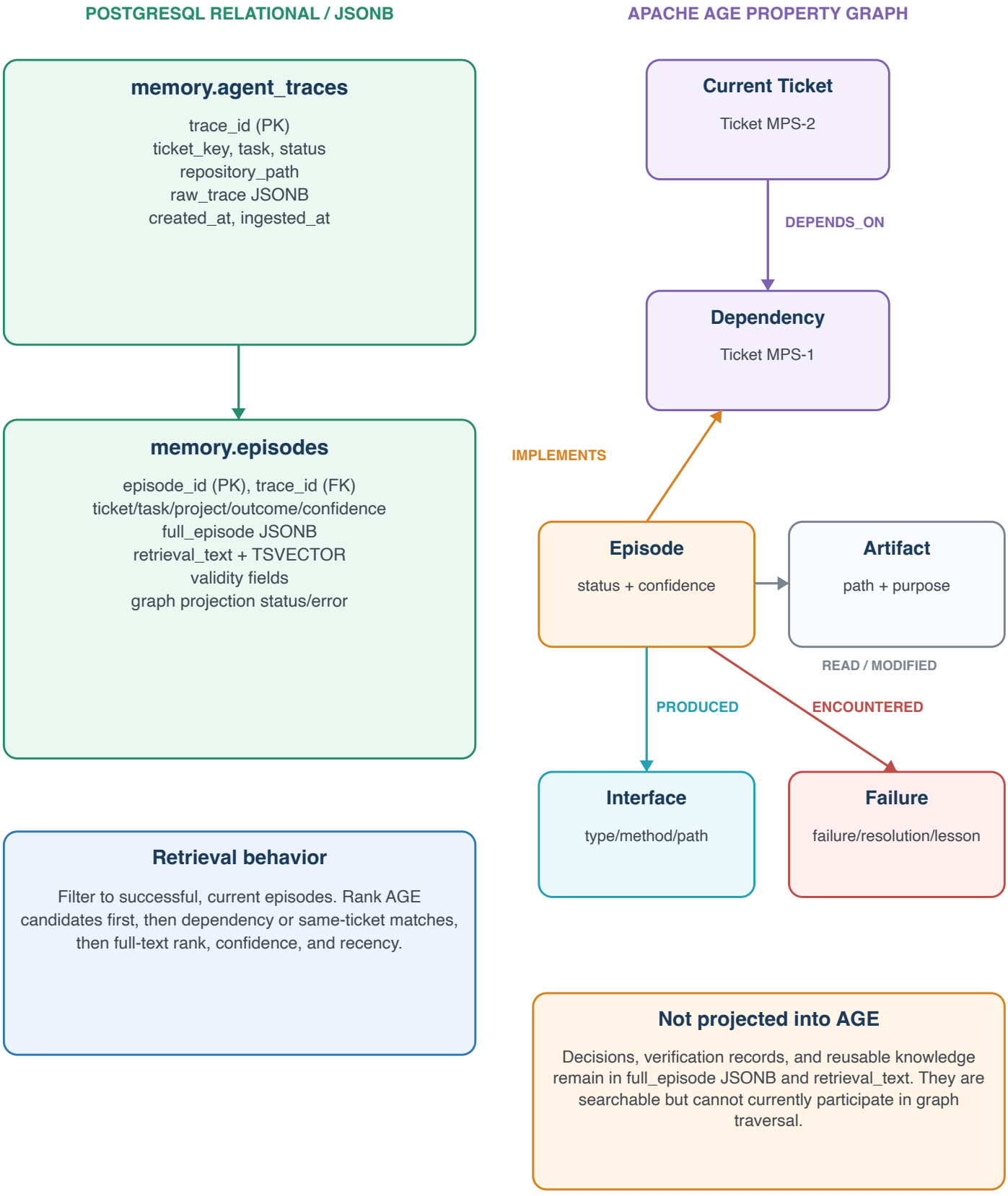
End-to-End Data Flow

Read top-to-bottom within each column; the right column feeds the next run back into step 2.



Canonical Storage and AGE Projection

PostgreSQL owns complete episode content; AGE contains selected relationships used for structural retrieval.



Retrieval behavior

Filter to successful, current episodes. Rank AGE candidates first, then dependency or same-ticket matches, then full-text rank, confidence, and recency.

Not projected into AGE

Decisions, verification records, and reusable knowledge remain in full_episode JSONB and retrieval_text. They are searchable but cannot currently participate in graph traversal.

Code Ownership Map

Multiple implementations coexist. Their episode schemas and PostgreSQL schemas are not interchangeable.

File / component	Role	Data responsibility
opencode_orchestrator_postgres_age.py	Primary closed loop	DB init; hybrid retrieval; plan/build tracing; LLM episode extraction; memory.* persistence; AGE projection
opencode_orchestrator_with_episode_memory.py	Local-memory variant	Ranks and stores canonical episodes in .episodic_memory/episodes.jsonl
opencode_orchestrator.py	Trace-only variant	Runs plan/build, mirrors tool events to Langfuse, and exports trace JSON
episodic_memory/extract.py	Deterministic extractor	Builds a compact episode and typed graph projection without an extraction LLM
episodic_memory/postgres.py + schema.sql	Standalone importer	Stores into episodic.* tables and a separate AGE model
query_episode_memory.py	Legacy query CLI	Queries only the local JSONL store; does not query PostgreSQL or AGE
orchestrator.py	Short-term memory demo	LangGraph MemorySaver conversation example; not durable episodic memory
opencode.json	Automatic tracing	Enables OpenTelemetry and the Langfuse OpenCode observability plugin
docker-compose.age.yml	Infrastructure	Runs PostgreSQL with Apache AGE and a persistent Docker volume
jira/ and confluence/	Requirements fixtures	Current per-ticket/per-page JSON inputs; not the aggregate filenames expected by the orchestrators
parking/	Mall parking output	MPS-2 entry-ticket implementation
mps/	Naming collision	Master Production Schedule application, not Mall Parking System configuration

Architectural Findings and Priorities

These findings describe the repository as read; no runtime, database, or network verification was performed.

1. Input layout mismatch

load_jira_context() expects jira_tickets.json and build_episode_evidence() expects confluence_stories.json. The repository instead contains jira/MPS-*.json and confluence/CONF-MPS-*.json. Dependencies and requirements provenance can therefore be absent.

2. Competing canonical models

The monolithic PostgreSQL orchestrator writes memory.* with an LLM-defined Episode schema. The standalone episodic_memory package writes episodic.* using a deterministic but different schema and graph model.

3. Raw-trace durability depends on extraction

In the primary path, store_trace_and_episode() runs only after the episode LLM succeeds. A failed or unavailable extraction model leaves the raw trace in Langfuse/local JSON but not in memory.agent_traces.

4. Partial graph projection

AGE models ticket dependencies, artifacts, interfaces, and failures. Decisions, verification evidence, and reusable knowledge stay in JSONB/full-text storage and are invisible to graph traversal.

5. Sensitive-data boundary

The trace includes prompts and tool inputs/outputs. The LLM extraction prompt asks for secret exclusion, but the primary evidence builder truncates rather than deterministically redacting credentials before transmission or storage.

6. Domain naming collision

mps/ implements Master Production Scheduling while ticket IDs use MPS for Mall Parking System. parking/ contains the MPS-2 mall-parking implementation. This can mislead agents and retrieval ranking.

Recommended architectural direction

Choose one canonical Episode contract and one PostgreSQL schema, adapt ingestion to the actual fixture layout, persist raw traces before optional LLM extraction, and treat AGE as a derived projection whose status can be rebuilt from PostgreSQL.